

Tut 01 – Introducción a C++

Ingeniería en Inteligencia Artificial — Algoritmos y Estructura de Datos

2C 2026 · Docente: Ing. Magali Marijuan

- Tut 01 – Introducción a C++
- Objetivos de la clase
- ¿Por qué C++?
- Compilado vs interpretado
 - Compilar y ejecutar un programa
 - Flags de compilación de la materia
- Estructura básica de un programa en C++
 - Los `#include`
 - `using namespace std` vs `std::`
 - La función `main`
- Tipos primitivos y tipado estático
- Entrada y salida: `cout` , `cin` y `endl`
- Estructuras de control
 - `if` / `else if` / `else`
 - Ciclo `for`
 - Ciclo `while`
- Funciones
 - Paso por valor vs paso por referencia
 - Sobrecarga de funciones
- Arreglos, `std::vector` y `std::string`
 - Arreglos estáticos
 - Intro a `std::vector`
 - Intro a `std::string`
 - Accesos fuera de rango
- Introducción a objetos
 - ¿Qué es una clase y qué es un objeto?
 - Atributos y métodos
 - El constructor
 - El destructor
 - Cómo se organizan los archivos: `.h` y `.cpp`

Objetivos de la clase

- Entender por qué usamos C++ en la materia.
- Distinguir un lenguaje compilado de uno interpretado (repaso desde Python).
- Aprender la estructura básica de un programa en C++ y cómo compilarlo con `g++`.
- Conocer los tipos primitivos, el tipado estático y las estructuras de control.
- Escribir funciones con paso por valor y por referencia, y entender la sobrecarga.
- Dar los primeros pasos con `std::vector` y `std::string` para trabajar con colecciones y texto.
- Dar una primera mirada a los objetos: qué es una clase, sus atributos y métodos, el constructor y el destructor, y cómo se organiza el código en archivos.

¿Por qué C++?

En la materia usamos C++ porque nos da control fino sobre la memoria y, al mismo tiempo, un montón de herramientas para escribir programas cómodos y ordenados. Es un lenguaje que nos deja ver "qué pasa por debajo" (dónde vive cada dato, cuándo se reserva y se libera la memoria) sin renunciar a estructuras de alto nivel.

Algunas de las herramientas que vamos a aprovechar:

- Programación orientada a objetos (`class`), que nos permite modelar nuestros propios tipos de datos.
- Una biblioteca estándar rica (`std::vector` , `std::string` , `std::map` , etc.).
- Manejo de entrada/salida con `cout` / `cin` .
- Referencias (`&`), una forma segura de pasar variables por "alias" sin usar punteros explícitos.

A lo largo de la materia vamos a combinar dos cosas: entender cómo funciona la memoria por dentro y usar las herramientas de alto nivel que ofrece el lenguaje.

Compilado vs interpretado

Como repaso: Python es un **lenguaje interpretado**. Un intérprete lee el código línea por línea y lo va traduciendo a código máquina en el momento de ejecutarlo.

C++ es un **lenguaje compilado**. Existe un **compilador** (en nuestro caso, `g++`) que traduce **todo** el código fuente a código máquina antes de ejecutarlo, generando un **archivo ejecutable**.

El código compilado es específico para una arquitectura y un sistema operativo.

Un ejecutable compilado en Linux no corre directamente en Windows ni en macOS.

	Interpretado (Python)	Compilado (C++)
¿Cuándo se traduce?	En tiempo de ejecución	En tiempo de compilación
¿Qué se genera?	Nada, se ejecuta directo	Un archivo ejecutable
Detección de errores	En tiempo de ejecución	Muchos errores se detectan antes de correr el programa
Velocidad de ejecución	Más lenta	Más rápida

Pregunta para pensar: si el compilador detecta errores de tipos antes de ejecutar el programa, ¿por qué en Python un error de tipos puede aparecer recién después de varios minutos de ejecución?

Compilar y ejecutar un programa

Para compilar un programa en C++ usamos el compilador `g++` :

```
g++ programa.cpp -o programa
```

Esto genera un archivo ejecutable llamado `programa` . Para ejecutarlo:

```
./programa
```

Importante: evitá compilar y correr el programa apretando el botón "Run" del editor. Siempre usá la consola (puede ser la consola integrada del editor). Así reproducís exactamente lo que va a hacer el corrector.

Flags de compilación de la materia

En la materia vamos a compilar siempre con flags estrictos, que hacen que el compilador sea mucho más exigente:

```
g++ -g -std=c++17 -Wall -Wextra -Wconversion -Werror -o programa programa.cpp
```

- `-g` : genera información de debug para poder usar un debugger (por ejemplo GDB).
- `-std=c++17` : fuerza el estándar C++17.
- `-Wall` : activa la mayoría de las advertencias importantes.
- `-Wextra` : activa advertencias adicionales que `-Wall` no incluye.
- `-Wconversion` : advierte sobre conversiones implícitas de tipos que pueden perder información.
- `-Werror` : convierte **todas** las advertencias en errores. Si hay un solo warning, no compila.

Tip: acostúmbrate a compilar con estos flags desde el día uno. Un programa que "compila" sin ellos puede estar lleno de warnings que esconden bugs.

Estructura básica de un programa en C++

```
#include <iostream>

int main() {
    std::cout << "Hola mundo" << std::endl;
    return 0;
}
```

Vamos a desarmar esta estructura pieza por pieza.

Los `#include`

La línea `#include <iostream>` le dice al compilador que incluya la biblioteca estándar de entrada/salida, que nos da acceso a `std::cout` , `std::cin` , etc.

Cada vez que queramos usar una herramienta de la biblioteca estándar, vamos a tener que incluir el archivo correspondiente con `#include` .

Necesito...	<code>#include</code>
Entrada/salida (<code>cout</code> , <code>cin</code>)	<code><iostream></code>
<code>std::string</code>	<code><string></code>
<code>std::vector</code>	<code><vector></code>
Funciones matemáticas (<code>pow</code> , <code>sqrt</code>)	<code><cmath></code>

using namespace std vs std::

Toda la biblioteca estándar de C++ vive dentro de un **namespace** (espacio de nombres) llamado `std`. Por eso, para usar `cout` tenemos dos opciones:

```
#include <iostream>

int main() {
    std::cout << "Hola" << std::endl; // usando std:: explícito
    return 0;
}
```

o bien:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hola" << endl; // sin std::
    return 0;
}
```

Tip: en programas chicos `using namespace std;` es cómodo, pero en la materia recomendamos acostumbrarse a escribir `std::` explícito. Ayuda a entender de dónde viene cada cosa y evita conflictos de nombres en programas más grandes.

La función `main`

Todo programa en C++ necesita una función `main`, que es el **punto de entrada** del programa: es la primera función que se ejecuta.

```
int main() {
    // acá va el código
    return 0;
}
```

El `return 0;` indica que el programa terminó **sin errores**. Cualquier otro valor de retorno se interpreta como un código de error.

Pregunta para pensar: ¿qué pasa si mi `main` no tiene ningún `return`? (Probalo y fijate qué dice el compilador.)

Tipos primitivos y tipado estático

C++ es un lenguaje de **tipado estático**: el tipo de cada variable se fija en tiempo de compilación y no puede cambiar durante la ejecución. (Esto es distinto de Python, donde una misma variable puede pasar de guardar un número a guardar un texto.)

```
int edad = 21;
edad = "veintiuno"; // ERROR de compilación
```

Algunos tipos primitivos comunes:

Tipo	Uso principal	Tamaño típico
bool	Verdadero / falso	1 byte
char	Caracteres	1 byte
int	Enteros	4 bytes
long	Enteros grandes	8 bytes
float	Decimales	4 bytes
double	Decimales de mayor precisión	8 bytes
unsigned int	Enteros sin signo (solo positivos)	4 bytes

```
int edad = 21;
double promedio = 8.5;
char inicial = 'M';
bool aprobado = true;
```

El tamaño exacto en bytes depende de la arquitectura y el compilador; los valores de la tabla son los más comunes en sistemas de 64 bits.

Entrada y salida: cout , cin y endl

Para imprimir por pantalla usamos `std::cout` junto con el operador `<<` :

```
#include <iostream>

int main() {
    std::cout << "Hola mundo" << std::endl;
    std::cout << "El número es: " << 10 << std::endl;
    return 0;
}
```

- `<<` es el operador de **inserción**: "inserta" lo que está a la derecha en el flujo de salida.
- `std::endl` inserta un salto de línea y además "vacía" el buffer de salida.

Para leer datos desde la consola usamos `std::cin` junto con el operador `>>`:

```
#include <iostream>

int main() {
    int edad;

    std::cout << "Ingrese su edad: ";
    std::cin >> edad;

    std::cout << "Edad: " << edad << std::endl;
    return 0;
}
```

- `>>` es el operador de **extracción**: "extrae" un valor del flujo de entrada y lo guarda en la variable.

Con `cin` no hace falta pasar la dirección de memoria de la variable: `cin` ya sabe cómo modificarla directamente porque trabaja con referencias (¡más sobre esto en la próxima clase!).

Pregunta para pensar: ¿qué pasa si le pido a `cin` que lea un `int` y el usuario escribe una palabra?

Estructuras de control

if / else if / else

```
#include <iostream>

int main() {
    int nota = 7;

    if (nota >= 9) {
        std::cout << "Excelente" << std::endl;
    } else if (nota >= 6) {
        std::cout << "Aprobado" << std::endl;
    } else {
        std::cout << "Desaprobado" << std::endl;
    }

    return 0;
}
```

Usamos los operadores de comparación (`==` , `!=` , `<` , `>` , `<=` , `>=`) y los operadores lógicos (`&&` para "y", `||` para "o", `!` para "no").

Ciclo for

```
#include <iostream>

int main() {
    for (int i = 0; i < 5; i++) {
        std::cout << i << std::endl;
    }
    return 0;
}
```

En Python el equivalente sería `for i in range(5): print(i)` . La diferencia clave es que en C++ el `for` expone explícitamente las tres partes: inicialización, condición de corte e incremento.

Ciclo `while`

```
#include <iostream>

int main() {
    int i = 0;
    while (i < 5) {
        std::cout << i << std::endl;
        i++;
    }
    return 0;
}
```

Tip: usá `for` cuando sabés de antemano cuántas veces querés iterar, y `while` cuando la condición de corte depende de algo que se calcula durante el ciclo.

Funciones

Una función en C++ necesita tipar tanto lo que devuelve como cada uno de sus parámetros:

```
int sumar(int a, int b) {
    return a + b;
}
```

Si la función no devuelve nada, el tipo de retorno es `void`.

Paso por valor vs paso por referencia

Por defecto, en C++ los parámetros se pasan **por valor**: la función recibe una **copia** del argumento, y modificarla dentro de la función no afecta a la variable original.

```
#include <iostream>

void incrementar(int x) {
    x = x + 1; // solo modifica la copia local
}

int main() {
    int numero = 5;
    incrementar(numero);
    std::cout << numero << std::endl; // imprime 5, no cambi 
    return 0;
}
```

C++ agrega las **referencias** (&), que permiten pasar una variable "por alias": el par metro pasa a ser **otro nombre** para la misma variable, sin necesidad de trabajar con punteros.

```
#include <iostream>

void incrementar(int &x) {
    x = x + 1; // modifica la variable original
}

int main() {
    int numero = 5;
    incrementar(numero);
    std::cout << numero << std::endl; // imprime 6
    return 0;
}
```

Forma de pasaje	Sintaxis	�La funci�n puede modificar el original?	Costo
Por valor	<code>void f(int x)</code>	No	Copia el dato (m�s caro si el tipo es grande)
Por referencia	<code>void f(int &x)</code>	S�	No copia, solo un alias
Por referencia constante	<code>void f(const int &x)</code>	No (protegida)	No copia, ideal para tipos grandes que no se van a modificar

Pregunta para pensar: si quiero pasar un `std::string` muy largo a una funci n que **no** necesita modificarlo,  qu  forma de pasaje me conviene usar? (Pista: pens  en el costo de

Sobrecarga de funciones

C++ permite definir varias funciones con el **mismo nombre**, siempre que se puedan distinguir por la cantidad o el tipo de sus parámetros. Esto se llama **sobrecarga** (*overloading*).

```
#include <iostream>

int sumar(int a, int b) {
    return a + b;
}

double sumar(double a, double b) {
    return a + b;
}

int main() {
    std::cout << sumar(2, 3) << std::endl;    // usa la versión con int
    std::cout << sumar(2.5, 3.5) << std::endl; // usa la versión con double
    return 0;
}
```

El compilador elige automáticamente qué versión usar según los tipos de los argumentos que le pasamos.

Este tipo de decisiones se toma en tiempo de compilación, no en tiempo de ejecución: por eso es fundamental que el tipado sea estático.

Arreglos, `std::vector` y `std::string`

Arreglos estáticos

En C++ existen los arreglos de tamaño fijo:

```
int numeros[5] = {1, 2, 3, 4, 5};

for (int i = 0; i < 5; i++) {
    std::cout << numeros[i] << std::endl;
}
```

El tamaño se define en tiempo de compilación y el arreglo vive en el **stack**.

Intro a `std::vector`

C++ agrega `std::vector`, un arreglo **dinámico** de la biblioteca estándar que puede crecer y achicarse en tiempo de ejecución:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> numeros = {1, 2, 3, 4, 5};

    numeros.push_back(6); // agrega un elemento al final

    for (int i = 0; i < numeros.size(); i++) {
        std::cout << numeros[i] << std::endl;
    }

    return 0;
}
```

- `numeros.size()` devuelve la cantidad de elementos que tiene el vector en ese momento.
- `numeros.push_back(x)` agrega `x` al final, redimensionando el vector si hace falta.

Vamos a ver `std::vector` en profundidad más adelante, cuando entendamos cómo funciona la memoria dinámica por dentro.

Intro a `std::string`

De la misma forma, `std::string` es el tipo de la biblioteca estándar para representar texto de forma cómoda:

```
#include <iostream>
#include <string>

int main() {
    std::string nombre = "Ada";
    std::cout << "Hola, " << nombre << std::endl;
    std::cout << "Tu nombre tiene " << nombre.size() << " letras" << std::endl;
    return 0;
}
```

Accesos fuera de rango

Tanto en arreglos como en `std::vector`, el operador `[]` **no verifica los límites**. Acceder a una posición fuera de rango es **comportamiento indefinido**:

```
int numeros[5] = {1, 2, 3, 4, 5};
std::cout << numeros[10] << std::endl; // comportamiento indefinido
```

Esto significa que el programa puede: parecer funcionar igual, mostrar un valor "basura", o directamente terminar con un error (`segmentation fault`).

Pregunta para pensar: `std::vector` tiene un método `.at(i)` que sí verifica los límites y lanza una excepción si el índice es inválido. ¿Por qué creen que `operator[]` no lo hace por defecto?

Introducción a objetos

Hasta acá vimos tipos que ya vienen dados: `int`, `double`, `std::vector`, `std::string`. Pero muchas veces queremos representar cosas de nuestro propio problema: un punto en el plano, una cuenta bancaria, un alumno. Para eso, C++ nos deja definir **nuestros propios tipos** usando **clases**.

¿Qué es una clase y qué es un objeto?

Una **clase** es como un "molde": describe qué datos guarda y qué operaciones se pueden hacer sobre ese tipo. Un **objeto** es una **instancia** concreta de esa clase, es decir, un valor construido a partir de ese molde.

Una analogía: la clase es el plano de una casa; los objetos son las casas concretas que construimos a partir de ese plano. Todas comparten la estructura, pero cada una tiene sus propios valores.

Atributos y métodos

Una clase combina dos cosas:

- **Atributos**: los datos que guarda cada objeto (por ejemplo, las coordenadas `x` e `y` de un punto).
- **Métodos**: las funciones que "viven adentro" de la clase y operan sobre esos atributos (por ejemplo, calcular la distancia al origen).

```

#include <iostream>
#include <cmath>

class Punto {
public:
    double x;
    double y;

    double distanciaAlOrigen() {
        return std::sqrt(x * x + y * y);
    }
};

int main() {
    Punto p;    // creamos un objeto de tipo Punto
    p.x = 3.0;  // accedemos a sus atributos con el punto
    p.y = 4.0;

    std::cout << "Distancia: " << p.distanciaAlOrigen() << std::endl; // 5
    return 0;
}

```

- Con el operador `.` accedemos a los atributos (`p.x`) y llamamos a los métodos (`p.distanciaAlOrigen()`) de un objeto.
- La palabra `public:` indica que esos atributos y métodos se pueden usar desde afuera de la clase. Por ahora vamos a trabajar con todo `public` ; más adelante en la materia van a ver cómo "esconder" datos con `private` .

El constructor

Fijarse que arriba tuvimos que crear el `Punto` y después setear `x` e `y` a mano. El **constructor** es un método especial que se ejecuta **automáticamente al crear el objeto** y nos permite inicializar sus atributos en un solo paso. Tiene el **mismo nombre que la clase** y no declara tipo de retorno.

```

#include <iostream>
#include <cmath>

class Punto {
public:
    double x;
    double y;

    Punto(double xInicial, double yInicial) { // constructor
        x = xInicial;
        y = yInicial;
    }

    double distanciaAlOrigen() {
        return std::sqrt(x * x + y * y);
    }
};

int main() {
    Punto p(3.0, 4.0); // el constructor inicializa x e y
    std::cout << "Distancia: " << p.distanciaAlOrigen() << std::endl; // 5
    return 0;
}

```

El constructor se ejecuta una sola vez, en el momento en que se crea el objeto.

El destructor

Así como el constructor se ejecuta al crear el objeto, el **destructor** se ejecuta **automáticamente cuando el objeto deja de existir** (por ejemplo, al terminar la función donde vive). Sirve para hacer "tareas de limpieza".

El destructor también tiene el mismo nombre que la clase, pero empieza con `~` y no recibe parámetros:

```
#include <iostream>

class Punto {
public:
    double x;
    double y;

    Punto(double xInicial, double yInicial) { // constructor
        x = xInicial;
        y = yInicial;
        std::cout << "Se creó un Punto" << std::endl;
    }

    ~Punto() { // destructor
        std::cout << "Se destruyó un Punto" << std::endl;
    }
};

int main() {
    Punto p(3.0, 4.0);
    std::cout << "Usando el punto..." << std::endl;
    return 0;
} // acá, al terminar main, se ejecuta el destructor de p
```

Este programa imprime:

```
Se creó un Punto
Usando el punto...
Se destruyó un Punto
```

Por ahora nuestros destructores no van a hacer mucho, pero cuando una clase administra recursos (por ejemplo, memoria que pide durante su vida), el destructor es el lugar donde se devuelven esos recursos. Vamos a volver sobre esto en la próxima clase.

Cómo se organizan los archivos: `.h` y `.cpp`

En los ejemplos de arriba pusimos todo en un solo archivo para simplificar. En la materia, sin embargo, vamos a **separar** cada clase en dos archivos:

- Un archivo de **encabezado** (`.h`), que contiene la **declaración** de la clase: qué atributos y métodos tiene. Es el "índice" de la clase.
- Un archivo de **implementación** (`.cpp`), que contiene el **cuerpo** de los métodos: qué hace cada uno por dentro.

punto.h — la declaración:

```
#ifndef PUNTO_H
#define PUNTO_H

class Punto {
public:
    double x;
    double y;

    Punto(double xInicial, double yInicial);
    double distanciaAlOrigen();
};

#endif
```

punto.cpp — la implementación:

```
#include "punto.h"
#include <cmath>

Punto::Punto(double xInicial, double yInicial) {
    x = xInicial;
    y = yInicial;
}

double Punto::distanciaAlOrigen() {
    return std::sqrt(x * x + y * y);
}
```

main.cpp — el programa que usa la clase:

```
#include <iostream>
#include "punto.h"

int main() {
    Punto p(3.0, 4.0);
    std::cout << "Distancia: " << p.distanciaAlOrigen() << std::endl;
    return 0;
}
```

Algunas cosas para notar:

- En el `.cpp` cada método lleva adelante el nombre de la clase y `::` (por ejemplo `Punto::distanciaAlOrigen`). Eso le dice al compilador "este método pertenece a la clase `Punto`".
- Para usar la clase desde otro archivo alcanza con hacer `#include "punto.h"` (con comillas, porque es un archivo nuestro y no de la biblioteca estándar).
- Las líneas `#ifndef PUNTO_H` / `#define PUNTO_H` / `#endif` son un **include guard**: evitan que el mismo encabezado se incluya dos veces por error. Por ahora alcanza con copiar el patrón usando el nombre del archivo en mayúsculas.

Para compilar un programa con varios `.cpp` los pasamos todos juntos a `g++` :

```
g++ -g -std=c++17 -Wall -Wextra -Wconversion -Werror -o programa main.cpp punto.cpp
```

Separar declaración e implementación puede parecer molesto al principio, pero mantiene el código ordenado: leyendo solo el `.h` uno entiende **qué** hace una clase, sin necesidad de leer **cómo** lo hace.